Arquitectura Event-Driven multilenguaje con RabbitMQ y Docker

Principios de Arquitectura de Software — Taller en grupo

Integrantes: David Esteban Suarez Lozano (2221909) · Thomas Alejandro Perez Rojas (2221906)
Jesus David Santodomingo Carrascal (2221922) — Fecha: 28 de octubre de 2026
Repositorio: https://github.com/Tapr16/taller-eventdriven-rabbitmq

1. Descripción del sistema

Simulamos los eventos de una tienda en línea. Cuatro productores generan eventos de dominio y cuatro consumidores reaccionan a ellos sin conocerse entre sí: el único punto de contacto es el broker RabbitMQ (AMQP 0-9-1). Cada servicio corre en su propio contenedor Docker y se compila dentro de su Dockerfile (multi-stage para Java y Go), por lo que solo se requiere Docker en cada nodo.

Servicio	Lenguaje / librería	Rol	Routing keys publicadas / bindings
productor-ordenes	Python / pika	Productor	orden.creada
productor-pagos	Java / amqp-client	Productor	pago.aprobado, pago.rechazado
productor-inventario	Node.js / amqplib	Productor	inventario.actualizado, inventario.agotado
productor-envios	Go / amqp091-go	Productor	envio.despachado, envio.entregado
consumidor-notificaciones	Python / pika	Consumidor	q.notificaciones ← orden.creada, envio.*
consumidor-facturacion	Java / amqp-client	Consumidor	q.facturacion ← pago.aprobado, pago.rechazado
consumidor-auditoria (+UI web)	Node.js / amqplib	Consumidor	q.auditoria ← # (todos los eventos)
consumidor-analitica	Go / amqp091-go	Consumidor	q.analitica ← orden.*, inventario.*, pago.*

Topología AMQP. Un exchange de tipo topic y durable (tienda.eventos); la routing key es el tipo de evento (dominio.acción). Cada consumidor declara su propia cola durable y sus bindings con comodines (* = una palabra, # = cero o más), de modo que un mismo evento llega a varias colas (p. ej. orden.creada llega a notificaciones, auditoría y analítica): publish/subscribe con filtrado por tema. Los mensajes son JSON persistentes (delivery_mode=2) con un campo nodo que identifica la máquina de origen; los consumidores usan ack manual y prefetch. Todos los servicios reintentan la conexión si el broker aún no está disponible.

2. Diagramas arquitecturales

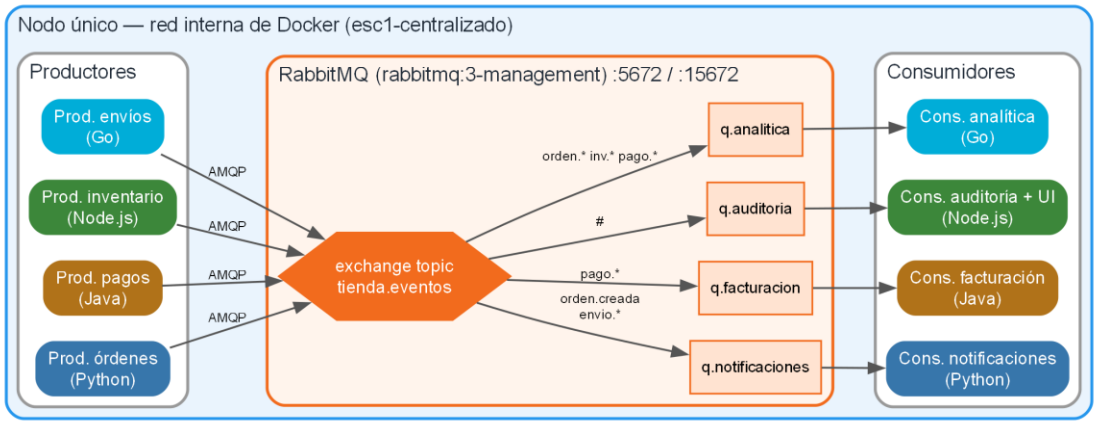


Figura 1. Escenario 1 — centralizado: broker, productores y consumidores en un solo nodo (1 docker-compose.yml; los servicios encuentran al broker por el nombre «rabbitmq» en la red interna de Docker).

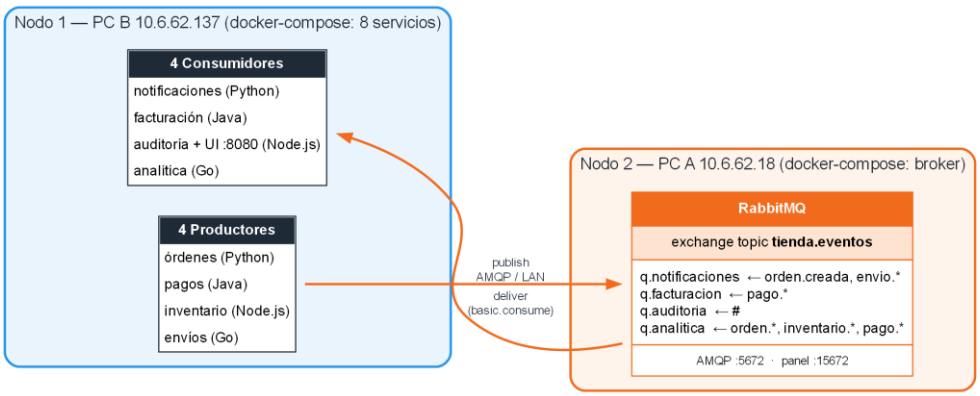


Figura 2. Escenario 2 — 2 nodos: Nodo 1 con los 8 servicios y Nodo 2 con el broker (2 docker-compose.yml; la IP del broker se pasa por .env → BROKER_IP).

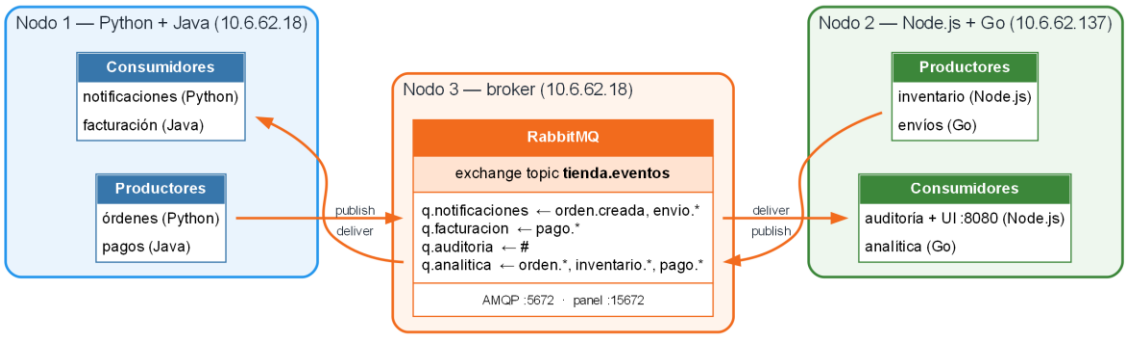


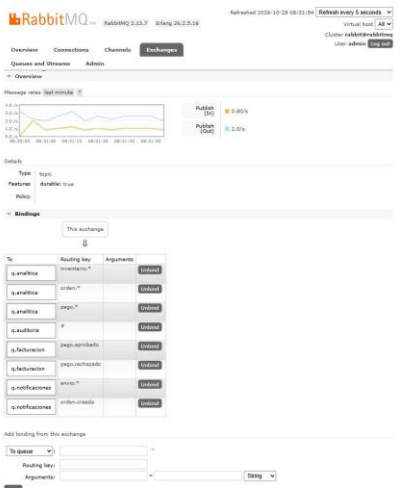
Figura 3. Escenario 3 — 3 nodos: Python+Java en el Nodo 1, Node.js+Go en el Nodo 2 y el broker en el Nodo 3 (3 docker-compose.yml). Los eventos entre nodos 1 y 2 siempre pasan por el broker.

3. Evidencias de ejecución

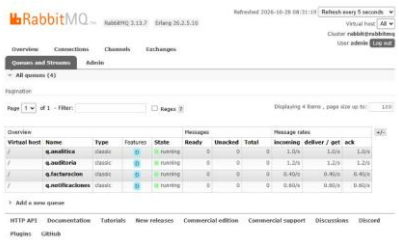
Todas las capturas y logs completos están en la carpeta evidencias/ del repositorio (github.com/Tapr16/taller-eventdriven-rabbitmq). Se usaron dos PCs de la sala: PC A 10.6.62.18 y PC B 10.6.62.137.

Escenario 1 — Centralizado

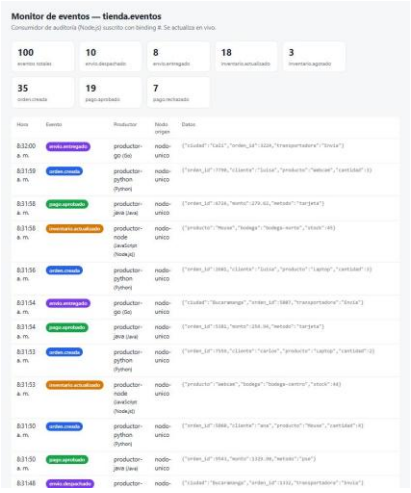
docker compose ps muestra 9 contenedores (broker + 8 servicios). En el panel se ven 8 conexiones AMQP desde la red interna 172.18.0.x, el exchange tienda.eventos con sus 8 bindings y 4 colas durables con 1 consumidor y 0 mensajes pendientes. La tasa de entrega (~3 msg/s) supera a la de publicación (~1 msg/s) porque cada evento se copia a varias colas.



Exchange topic y bindings



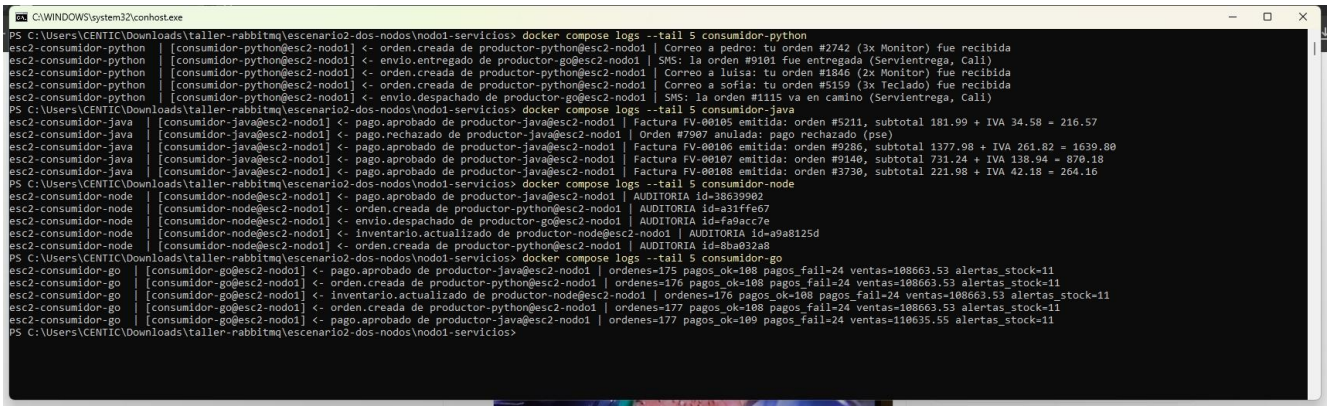
4 colas, 1 consumidor c/u



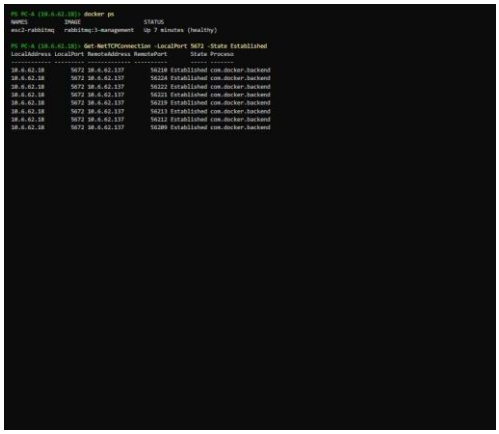
UI web de auditoría (:8080)

Escenario 2 — 2 nodos (PC B = Nodo 1 servicios, PC A = Nodo 2 broker)

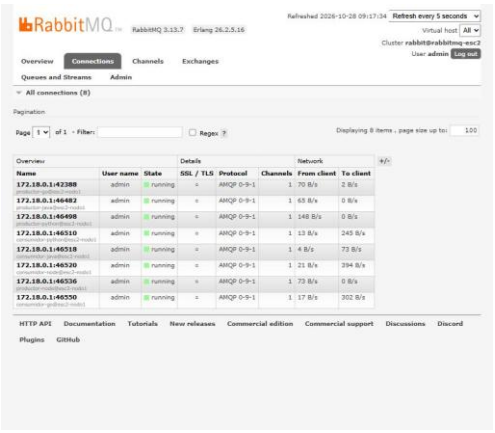
En el PC A solo corre esc2-rabbitmq y aparecen 8 conexiones TCP establecidas al puerto 5672 desde 10.6.62.137. El panel identifica cada conexión como servicio@esc2-nodo1, y en el PC B los 4 consumidores procesan los eventos:



Logs de los 4 consumidores en el Nodo 1 (PC B) recibiendo eventos a través del broker remoto.



PC A: conexiones TCP desde 10.6.62.137

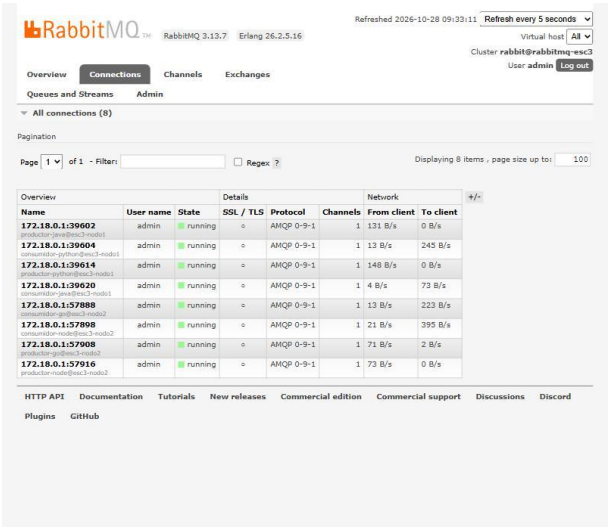


Panel: 8 conexiones @esc2-nodo1

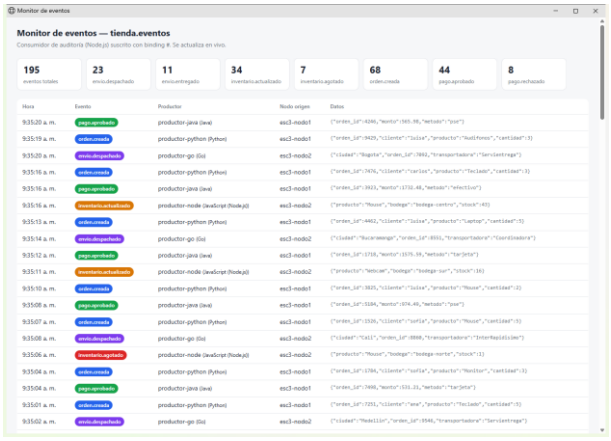
Escenario 3 — 3 nodos (PC A = Nodo 3 broker + Nodo 1, PC B = Nodo 2)

Por disponibilidad de equipos, el Nodo 1 y el broker compartieron máquina física, pero como proyectos compose separados y sin red Docker en común: el Nodo 1 se conecta por la IP de red (BROKER_IP=10.6.62.18), igual que el Nodo 2. El panel muestra 4 conexiones @esc3-nodo1 y 4 @esc3-nodo2, y los logs evidencian tráfico cruzado entre nodos:

```
[consumidor-go@esc3-nodo2] <- pago.aprobado de productor-java@esc3-nodo1 | ordenes=1 pagos_ok=1 ventas=977.22
[consumidor-python@esc3-nodo1] <- envio.entregado de productor-go@esc3-nodo2 | SMS: la orden #9218 fue entregada
```



Panel: conexiones @esc3-nodo1 y @esc3-nodo2



UI web en Nodo 2: eventos de ambos nodos

4. Comparación entre escenarios

Criterio	Esc. 1 — Centralizado	Esc. 2 — 2 nodos	Esc. 3 — 3 nodos
Configuración	Mínima: un compose; resolución por nombre de servicio (rabbitmq)	Hay que conocer la IP del broker (.env) y abrir el puerto 5672	Igual que el 2, más coordinación entre equipos
Latencia	Muy baja (red virtual local)	Añade un salto de red LAN	Igual que el 2; los eventos Nodo 1↔Nodo 2 pasan siempre por el Nodo 3
Escalabilidad	Limitada a los recursos de una máquina	El broker no compite por CPU/RAM con los servicios	Servicios repartidos; se agregan nodos sin tocar los existentes
Tolerancia a fallos	Cae el nodo → cae todo	Cae el Nodo 1 → los mensajes esperan en colas durables; cae el Nodo 2 → nadie se comunica	Cae el Nodo 1 o 2 → el otro sigue operando; el broker sigue siendo punto único de falla
Seguridad	Puertos solo locales	AMQP expuesto en la LAN: usuario propio (no guest); idealmente TLS	Igual que el 2
Depuración	Todos los logs en un solo lugar	Logs repartidos en 2 máquinas	Logs en 3 máquinas; el campo nodo y el nombre de conexión ayudan a rastrear

Ventajas del estilo event-driven: desacoplamiento total (movimos consumidores entre nodos sin modificar productores ni cambiar código, solo los docker-compose), interoperabilidad entre 4 lenguajes gracias a AMQP + JSON, y amortiguación: si un consumidor se detiene, sus mensajes se acumulan en la cola y se procesan al volver.

Limitaciones: el broker es un punto único de falla (se mitigaría con un clúster RabbitMQ y quorum queues), la consistencia es eventual y es más difícil seguir un flujo de punta a punta sin trazas distribuidas.

Retos técnicos encontrados

- El usuario guest de RabbitMQ solo acepta conexiones locales → se creó el usuario admin para los escenarios distribuidos.
- Los servicios pueden arrancar antes que el broker → reintento de conexión en los 4 lenguajes, y healthcheck + depends_on en el escenario 1.
- Docker Desktop publica el puerto con NAT, así que RabbitMQ ve a todos los clientes remotos como 172.18.0.1 → se nombró cada conexión AMQP (servicio@nodo) y se verificaron las IP reales con Get-NetTCPConnection en el host del broker.
- Firewall de Windows y red de la universidad al exponer el 5672 entre PCs; Docker Desktop fallaba en los PCs compartidos por sockets de sesiones anteriores (script arreglar-docker.ps1).
- Compilar Java y Go sin tenerlos instalados → multi-stage builds en los Dockerfile.

5. Conclusiones

- El mismo código y las mismas imágenes funcionaron sin cambios en los tres escenarios: solo cambió dónde corre cada contenedor y la dirección del broker. Esa es la principal ventaja del estilo event-driven con un broker: la topología física queda fuera del código.
- El exchange topic con bindings por comodines permitió que un mismo evento llegue a varios consumidores con intereses distintos, sin que el productor sepa quién lo consume.
- Distribuir mejora el aislamiento de recursos y la tolerancia a la caída de nodos de servicios, a cambio de más configuración de red, seguridad y dificultad de depuración; el broker centralizado sigue siendo el punto crítico, por lo que en producción se usaría un clúster.